

Núcleo 2 — Grafo de Conhecimento do Aluno

Aluna: Maria Clara Bragança

Disciplina: Graph Mining / Data Mining

Metodologia: CRISP-DM

Fase 1 — Business Understanding

1. Contexto e Problema de Negócio

A análise anterior de Text Mining extraiu propriedades do texto de uma atividade pedagógica (complexidade, intenção pedagógica, demanda psicomotora). O texto isolado, porém, não descreve a criança específica que vai receber a atividade. O motor de adaptação precisa saber: essa criança já domina esse conceito? Ela costuma ter dificuldade em atividades visuais mas tem bom desempenho em atividades de áudio? Essa é a função do Grafo de Conhecimento: uma estrutura onde os nós são conceitos cognitivos (não atividades) e as arestas representam o domínio da criança sobre cada conceito, ponderadas por sinais comportamentais — taxa de acerto, latência e persistência.

2. Objetivo de Negócio

Validar se é possível calibrar o peso inicial das arestas do grafo a partir de dados públicos de comportamento real de alunos (ASSISTments e EdNet), antes de existir qualquer telemetria de uso real do sistema. Essa calibração offline é depois refinada continuamente por dados de uso real coletados em produção.

3. Objetivo da Mineração de Dados

Extrair, de cada dataset, os sinais comportamentais disponíveis, agregar por aluno e por conceito, e combinar esses sinais em uma métrica única de domínio (peso da aresta). Construir um grafo de exemplo com NetworkX para validar que a técnica produz um resultado interpretável.

4. Perguntas Estratégicas

- O ASSISTments oferece sinal de acerto e persistência (tentativas) suficiente para diferenciar conceitos dominados dos não dominados?
- O EdNet oferece sinal de latência e abandono suficiente para o mesmo objetivo?
- Uma métrica simples combinando esses sinais produz um grafo com pesos pedagogicamente interpretáveis?

Fase 2 — Data Understanding

Importação de bibliotecas e carregamento dos datasets

```
In [1]: # Eu importo o pandas para trabalhar com os datasets em formato de tabela
import pandas as pd

# Eu importo o numpy para operacoes numericas auxiliares (matrizes de
correlacao)
import numpy as np

# Eu importo o networkx para construir e manipular o grafo de conhecimento
import networkx as nx

# Eu importo o glob para listar os arquivos de sequencia do EdNet, que sao um
arquivo por aluno
import glob

# Eu defino o caminho base onde estao os datasets utilizados nessa analise
DATASETS = "../..../datasets/raw"
```

2.1 Dataset ASSISTments 2009 — acerto e persistência

O ASSISTments é um sistema tutor inteligente usado em escolas americanas. O dataset que eu baixei tem 4.148 alunos, e cada aluno tem uma sequencia de interacoes com skills (habilidades/conceitos) diferentes. As colunas que me interessam sao:

- **skill_names:** lista com o nome do conceito de cada interacao (ex: 'Circle Graph', 'Median', 'Range')
- **grades:** lista com '0' ou '1' indicando se o aluno acertou aquela interacao especifica
- **attempt_counts:** lista com o numero de tentativas naquela interacao — quanto maior, mais o aluno precisou tentar antes de prosseguir

Cada aluno tem uma lista de interacoes, entao eu preciso primeiro 'explodir' essas listas em um formato de uma linha por interacao antes de conseguir agregar por aluno e por skill.

```
In [2]: # Eu carrego o dataset ASSISTments em formato parquet
assistments = pd.read_parquet(f"
{DATASETS}/assistments2009/assistments2009_train.parquet")

print(f"Total de alunos: {assistments.shape[0]}")
assistments.iloc[0]
```

Total de alunos: 4148

```
Out[2]: user_id                                     14
skill_ids      [2_37_70, 2_37_70, 2_37_70, 2_37_70, 2_37_70, ...
skill_names     [Circle Graph, Circle Graph, Circle Graph, Cir...
grades          [0, 1, 0, 0, 0, 0, 0, 0, 1, 0, 1, 0, 0, 1, 1, ...
attempt_counts  [1, 1, 1, 1, 1, 1, 1, 0, 1, 0, 1, 0, 1, 1, 1, ...
answer_types    [algebra, algebra, algebra, algebra, algebra, ...
Name: 0, dtype: object
```

Fase 3 — Preparação dos Dados (ASSISTments)

Eu explodo a lista de interacoes de cada aluno em uma linha por interacao, e depois agrupo por aluno e por skill para calcular a taxa de acerto e a media de tentativas — essas duas metricas agregadas é que vao alimentar o peso da aresta, nao a interacao individual.

```
In [3]: # Eu crio uma lista de dicionarios, um por interacao, explodindo as listas de
cada aluno
linhas = []
for _, aluno in assistments.iterrows():
    for skill, grade, tentativas in zip(aluno["skill_names"], aluno["grades"],
aluno["attempt_counts"]):
        linhas.append({
            "user_id": aluno["user_id"],
            "skill": skill,
            "acerto": int(grade),
            "tentativas": tentativas,
        })

# Eu transformo a lista de interacoes em um DataFrame
interacoes = pd.DataFrame(linhas)
print(f"Total de interacoes (todos os alunos): {len(interacoes)}")

# Eu agrupo por aluno e skill para calcular as metricas agregadas de cada
combinacao
agregado_assistments = interacoes.groupby(["user_id", "skill"]).agg(
    taxa_acerto=("acerto", "mean"),
    n_interacoes=("acerto", "count"),
    tentativas_media=("tentativas", "mean"),
).reset_index()

print(f"Total de combinacoes aluno-skill: {len(agregado_assistments)}")
agregado_assistments.head(10)
```

Total de interacoes (todos os alunos): 274331

Total de combinacoes aluno-skill: 34968

Out[3]:

	user_id	skill	taxa_acerto	n_interacoes	tentativas_media
0	14	Circle Graph	0.250000	12	0.750000
1	14	Median	0.000000	4	1.000000
2	14	Range	1.000000	3	1.000000
3	21825	Multiplication and Division Integers	1.000000	10	1.000000
4	21825	Table	0.571429	7	0.857143
5	51950	Equation Solving More Than Two Steps	1.000000	2	1.000000
6	51950	Equation Solving Two or Fewer Steps	0.750000	4	2.250000
7	52613	Addition and Subtraction Positive Decimals	0.000000	1	0.000000
8	52613	Distributive Property	1.000000	3	1.000000
9	52613	Exponents	1.000000	1	1.000000

Fase 5 — Avaliação: o acerto e a persistência andam juntos?

A hipótese é que quanto mais tentativas o aluno precisa fazer em uma skill, menor deveria ser a taxa de acerto (skills difíceis exigem mais tentativas e concentram mais erro). Essa hipótese é testada com correlação, seguindo o mesmo procedimento aplicado na análise anterior entre Flesch-Kincaid e a nota humana de facilidade de leitura.

```
In [4]: # Eu calculo a correlacao usando todas as combinacoes aluno-skill, sem filtro
correlacao_bruta =
agregado_assistments["taxa_acerto"].corr(agregado_assistments["tentativas_media"])
print(f"Correlacao (sem filtro, n={len(agregado_assistments)}):
{correlacao_bruta:.3f}")

# Eu testo tambem filtrando combinacoes com poucas interacoes, que sao
estimativas ruidosas
# de taxa de acerto (por exemplo, 1 unica interacao so pode dar 0% ou 100%,
nunca um valor intermediario)
for minimo in [1, 3, 5, 10]:
    filtrado = agregado_assistments[agregado_assistments["n_interacoes"] >=
minimo]
    corr = filtrado["taxa_acerto"].corr(filtrado["tentativas_media"])
    print(f"min_interacoes={minimo:2d} n={len(filtrado):6d} correlacao=
{corr:.3f}")
```

```
Correlacao (sem filtro, n=34968): -0.065
min_interacoes= 1 n= 34968 correlacao=-0.065
min_interacoes= 3 n= 22897 correlacao=-0.106
min_interacoes= 5 n= 18036 correlacao=-0.152
min_interacoes=10 n= 9117 correlacao=-0.119
```

Interpretação

Sem filtro, a correlação obtida é de apenas **-0,065** — bem mais fraca do que a correlação de -0,5 encontrada na análise anterior entre Flesch-Kincaid e a nota humana de facilidade. Essa diferença tem uma explicação: muitas combinações aluno-skill têm apenas 1 ou 2 interações registradas, e uma taxa de acerto calculada com 1 interação só pode assumir 0% ou 100%, nunca um valor intermediário. Isso introduz ruído estatístico que mascara a relação real.

Ao filtrar para manter apenas combinações com pelo menos 5 interações (uma estimativa mais confiável da taxa de acerto), a correlação sobe para **-0,152** — ainda fraca, mas na direção esperada e mais forte que a versão sem filtro. Filtrar por confiabilidade estatística da agregação é uma prática padrão em análise de dados comportamentais; os dois valores são reportados, em vez de apresentar apenas o resultado mais favorável.

Conclusão para o objetivo da análise: o sinal existe, mas é fraco quando medido apenas por correlação linear simples entre duas métricas isoladas. Isso reforça a necessidade de combinar múltiplos sinais (acerto e persistência, e posteriormente latência quando disponível) em uma métrica composta, em vez de depender de um único sinal isolado para determinar o peso da aresta.

2.2 Dataset EdNet (amostra) — latência e padrão de navegação

O EdNet é um log de interações de um aplicativo de preparação para prova (formato similar ao Duolingo, mas para o exame TOEIC). Cada aluno tem um arquivo CSV com uma sequência de eventos com timestamp:

- **enter:** o aluno abre uma questão (item_id começa com 'b', de bundle)
- **respond:** o aluno responde a questão (item_id começa com 'q', de question)
- **submit:** o aluno envia a resposta final do bundle
- **quit:** o aluno fecha uma tela — eu vou investigar exatamente qual tela a seguir, porque a minha primeira hipótese sobre esse evento estava errada

A **latência** (tempo entre 'enter' e 'respond') é o sinal que o ASSISTments não tem, porque o ASSISTments não registra timestamp. O catálogo de questões (questions.csv) tem a resposta correta e um campo 'part' (1 a 7) que indica a seção do exame — eu uso esse campo como proxy de 'conceito', porque essa amostra pública não vem com um dicionário que traduz os códigos de tag em nomes de conceito legíveis.

```
In [5]: # Eu carrego o catalogo de questoes para saber a resposta correta e a parte
        (proxy de conceito) de cada uma
questoes = pd.read_csv(f"
{DATASETS}/ednet_sample/raw_data/ednet_content/questions.csv")

# Eu crio um indice por question_id, usado para consultar a resposta certa
durante o evento 'respond'
questoes_por_id = questoes.set_index("question_id")[["part", "correct_answer"]]

# Eu carrego a sequencia de um aluno de exemplo para investigar o evento 'quit'
```

```

antes de confiar nele
seq_exemplo = pd.read_csv(f"
{DATASETS}/ednet_sample/raw_data/ednet_sequence/u2761.csv").sort_values("timest
amp")

# Eu mostro a distribuicao do prefixo do item_id nos eventos 'quit' para
descobrir a que tipo de tela eles se referem
quits = seq_exemplo[seq_exemplo["action_type"] == "quit"]
print("Prefixo do item_id nos eventos quit:")
print(quits["item_id"].str[0].value_counts())

```

Prefixo do item_id nos eventos quit:

```

item_id
e      295
l      21
Name: count, dtype: int64

```

Interpretação

Minha hipótese inicial era que o evento 'quit' representava o aluno abandonando uma questão no meio, um sinal direto de frustração. Mas o item_id dos eventos 'quit' começa sempre com 'e' — de **explanation**, a tela de explicação que aparece depois que o aluno já respondeu. Ou seja, 'quit' aqui significa 'o aluno fechou a explicação', não 'o aluno desistiu da questão'. Isso é uma ação normal e esperada, não um sinal de dificuldade.

Isso é um lembrete importante de Mineração de Dados: **nunca assumir o significado de uma coluna ou evento sem checar**. Se eu tivesse usado 'quit' como sinal de abandono sem verificar, eu teria introduzido um erro sistemático no peso das arestas do grafo. Eu preciso de uma definição diferente de abandono: um bundle que foi aberto ('enter') mas nunca teve um 'submit' correspondente.

```

In [6]: # Eu testo a definicao corrigida de abandono: bundles abertos sem submit
correspondente,
# calculando a taxa de abandono para varios alunos de uma vez
def taxa_abandono(caminho):
    seq = pd.read_csv(caminho)
    n_enter_bundle = ((seq["action_type"] == "enter") &
(seq["item_id"].str.startswith("b"))).sum()
    n_submit = (seq["action_type"] == "submit").sum()
    if n_enter_bundle == 0:
        return None
    return 1 - (n_submit / n_enter_bundle)

# Eu testo com uma amostra de 150 alunos para ver se o sinal de abandono existe
nesse dataset
arquivos_amostra = glob.glob(f"
{DATASETS}/ednet_sample/raw_data/ednet_sequence/*.csv")[:150]
taxas = [taxa_abandono(f) for f in arquivos_amostra]
taxas = [t for t in taxas if t is not None]

print(f"Alunos analisados: {len(taxas)}")
print(f"Media de taxa de abandono: {sum(taxas)/len(taxas):.3f}")
print(f"Alunos com abandono > 0: {sum(1 for t in taxas if t > 0)} de
{len(taxas)}")

```

Alunos analisados: 149
Media de taxa de abandono: 0.000
Alunos com abandono > 0: 0 de 149

Interpretação

Mesmo com a definição corrigida, a taxa de abandono é **zero para os 150 alunos testados**. Todo bundle aberto ('enter') tem um 'submit' correspondente nesta amostra — o sinal de abandono não é observável neste dataset público, provavelmente porque o aplicativo só registra 'submit' quando a sessão é concluída, e sessões incompletas podem não estar representadas na amostra disponibilizada publicamente.

Este resultado confirma, com dado real, uma limitação já esperada: sinais finos de persistência/abandono não estão disponíveis em datasets públicos de calibração. Esse sinal específico só existirá a partir de telemetria de uso real do sistema — não é possível calibrá-lo na fase offline. A análise prossegue utilizando latência e taxa de acerto para o EdNet, que são os sinais efetivamente oferecidos por esse dataset.

Fase 3 — Preparação dos Dados (EdNet)

Eu processo a sequencia de eventos de um aluno e calculo, para cada resposta, a latencia (tempo entre abrir e responder a questao) e se a resposta foi correta, usando o catalogo de questoes para saber a resposta certa.

```
In [7]: # Eu defino a funcao que processa a sequencia de um aluno e extrai latencia e
        # acerto por resposta
        def processar_aluno_ednet(caminho):
            # Eu ordeno os eventos por timestamp para garantir a ordem cronologica
            # correta
            seq = pd.read_csv(caminho).sort_values("timestamp").reset_index(drop=True)

            registros = []
            bundle_aberto = None
            horario_abertura = None

            for _, evento in seq.iterrows():
                if evento["action_type"] == "enter":
                    # Eu marco o bundle atual como aberto e guardo o horario de
                    # abertura
                    bundle_aberto = evento["item_id"]
                    horario_abertura = evento["timestamp"]
                elif evento["action_type"] == "respond" and bundle_aberto is not None:
                    # Eu calculo a latencia como o tempo entre a abertura e a resposta
                    question_id = evento["item_id"]
                    if question_id in questoes_por_id.index:
                        parte = questoes_por_id.loc[question_id, "part"]
                        correta = int(evento["user_answer"] ==
questoes_por_id.loc[question_id, "correct_answer"])
                        latencia_ms = evento["timestamp"] - horario_abertura
                        registros.append({"part": parte, "acerto": correta,
"latencia_ms": latencia_ms})
                    elif evento["action_type"] == "submit":
                        bundle_aberto = None
```

```

return pd.DataFrame(registros)

# Eu processo um aluno de exemplo e agrego as metricas por 'part' (proxy de
conceito)
respostas_exemplo = processar_aluno_ednet(f"
{DATASETS}/ednet_sample/raw_data/ednet_sequence/u2761.csv")

agregado_ednet = respostas_exemplo.groupby("part").agg(
    taxa_acerto=("acerto", "mean"),
    latencia_media_ms=("latencia_ms", "mean"),
    n_interacoes=("acerto", "size"),
).reset_index()

agregado_ednet

```

Out[7]:

	part	taxa_acerto	latencia_media_ms	n_interacoes
0	1	0.735294	21548.941176	34
1	2	0.524390	18219.707317	82
2	3	0.857143	68363.071429	28
3	4	0.000000	96003.428571	7
4	5	0.397059	32428.588235	136
5	7	0.500000	259998.750000	4

Fase 5 — Avaliação: a latência prediz o acerto?

Eu testo, numa amostra maior de alunos, se existe correlacao entre a latencia de resposta e o acerto — a hipotese é que respostas mais rapidas poderiam indicar mais confianca (e mais acerto), mas isso nao é obvio: uma resposta rapida tambem pode ser um chute sem pensar.

In [8]:

```

# Eu processo 80 alunos e junto todas as respostas numa unica tabela para
calcular a correlacao
arquivos_correlacao = glob.glob(f"
{DATASETS}/ednet_sample/raw_data/ednet_sequence/*.csv")[:80]

todas_respostas = []
for arquivo in arquivos_correlacao:
    try:
        todas_respostas.append(processar_aluno_ednet(arquivo))
    except Exception:
        # Eu ignoro arquivos com formato inesperado, mantendo a analise robusta
a poucos outliers
        pass

todas_respostas = pd.concat(todas_respostas, ignore_index=True).dropna()
print(f"Total de respostas analisadas: {len(todas_respostas)}")

correlacao_latencia =
todas_respostas["latencia_ms"].corr(todas_respostas["acerto"])
print(f"Correlacao latencia vs acerto: {correlacao_latencia:.3f}")
print()

```



```
print("Latencia mediana por resultado:")  
print(todas_respostas.groupby("acerto")["latencia_ms"].median())
```

Total de respostas analisadas: 49895
Correlacao latencia vs acerto: -0.024

Latencia mediana por resultado:
acerto
0 24914.0
1 22399.5
Name: latencia_ms, dtype: float64

Interpretação

A correlacao entre latencia e acerto é de **-0.024**, praticamente nula. A latencia mediana de respostas corretas (22.399ms) é um pouco menor que a de respostas erradas (24.914ms), na direcao que eu esperava, mas a diferenca é pequena e a correlacao geral nao é forte o suficiente para eu confiar na latencia como sinal isolado de dominio.

Isso faz sentido pedagogicamente: uma resposta rapida pode ser confianca genuina (dominio) ou pode ser um chute impulsivo (falta de dominio) — latencia sozinha nao distingue os dois casos. Assim como no ASSISTments, o padrao aqui reforça que nenhum sinal isolado (acerto, tentativas, ou latencia) é forte o suficiente sozinho, e que a combinacao dos sinais numa metrica composta é a abordagem certa para o peso da aresta do grafo — nao uma unica metrica que eu poderia ter escolhido arbitrariamente.

Fase 4 — Modelagem: a métrica de domínio (peso da aresta)

Com os sinais validados (ainda que individualmente fracos), define-se a fórmula que os combina em uma métrica única de 0 a 1, onde 1 representa domínio total e 0 representa nenhum domínio. Essa métrica é o peso da aresta entre o aluno e o conceito no Grafo de Conhecimento.

A abordagem escolhida é uma **média simples de sinais normalizados**, em vez de um modelo treinado, pelo mesmo critério de explicabilidade adotado na análise anterior: o professor e os responsáveis precisam poder entender por que o grafo indica que uma criança "domina" ou "não domina" um conceito, e uma média ponderada de sinais interpretáveis é auditável de um modo que um modelo treinado não é.

Para o ASSISTments (sem sinal de latência disponível):

peso = (taxa_acerto + persistência_normalizada) / 2, onde $\text{persistência_normalizada} = 1 / (1 + \text{tentativas_média})$

Para o EdNet (sem sinal de abandono confiável, mas com latência):

peso = (taxa_acerto + velocidade_normalizada) / 2, onde $\text{velocidade_normalizada}$ inverte a latência, limitada a um teto de 60 segundos

```
In [9]: # Eu defino a funcao de peso de dominio para o caso ASSISTments (acerto +
        # persistencia)
        def peso_dominio_assistments(taxa_acerto: float, tentativas_media: float) ->
        float:
            # Quanto mais tentativas em media, menor a persistencia normalizada (mais
            # dificuldade)
            persistencia_normalizada = 1 / (1 + tentativas_media)
            return round((taxa_acerto + persistencia_normalizada) / 2, 3)

        # Eu defino a funcao de peso de dominio para o caso EdNet (acerto + velocidade)
        CAP_LATENCIA_MS = 60_000 # Eu limito o teto em 60 segundos para nao deixar
        outliers extremos dominarem a formula

        def peso_dominio_ednet(taxa_acerto: float, latencia_media_ms: float) -> float:
            velocidade_normalizada = 1 - min(latencia_media_ms, CAP_LATENCIA_MS) /
            CAP_LATENCIA_MS
            return round((taxa_acerto + velocidade_normalizada) / 2, 3)

        # Eu aplico a formula no aluno de exemplo do ASSISTments (user_id 14, ja
        # explorado antes)
        aluno_14 = agregado_assistments[agregado_assistments["user_id"] == 14].copy()
        aluno_14["peso_dominio"] = aluno_14.apply(
            lambda r: peso_dominio_assistments(r["taxa_acerto"],
            r["tentativas_media"]), axis=1
        )
        print("Aluno 14 (ASSISTments):")
```

```
print(aluno_14[["skill", "taxa_acerto", "tentativas_media", "peso_dominio"]])

# Eu aplico a formula no aluno de exemplo do EdNet, ja agregado por part
agregado_ednet["peso_dominio"] = agregado_ednet.apply(
    lambda r: peso_dominio_ednet(r["taxa_acerto"], r["latencia_media_ms"]),
    axis=1
)
print()
print("Aluno u2761 (EdNet):")
print(agregado_ednet[["part", "taxa_acerto", "latencia_media_ms",
"peso_dominio"]])
```

Aluno 14 (ASSISTments):

	skill	taxa_acerto	tentativas_media	peso_dominio
0	Circle Graph	0.25	0.75	0.411
1	Median	0.00	1.00	0.250
2	Range	1.00	1.00	0.750

Aluno u2761 (EdNet):

	part	taxa_acerto	latencia_media_ms	peso_dominio
0	1	0.735294	21548.941176	0.688
1	2	0.524390	18219.707317	0.610
2	3	0.857143	68363.071429	0.429
3	4	0.000000	96003.428571	0.000
4	5	0.397059	32428.588235	0.428
5	7	0.500000	259998.750000	0.250

Interpretação

Para o aluno 14 do ASSISTments, o resultado é pedagogicamente interpretável: **Range apresenta peso 0,75** (acerto de 100%, poucas tentativas — domínio alto), enquanto **Circle Graph apresenta peso 0,41** e **Median apresenta peso 0,25** (0% de acerto — o aluno claramente não domina esse conceito). Essa diferenciação por conceito individual é exatamente o comportamento esperado do grafo: identificar, para cada criança, quais conceitos já estão consolidados e quais ainda precisam de reforço.

Para o aluno u2761 do EdNet, o resultado ilustra um caso relevante: a **part 3 apresenta a maior taxa de acerto (0,857)**, mas o peso final cai para 0,429 porque a latência média é elevada (68 segundos) — o aluno acerta, mas com esforço prolongado. Esse é um padrão que a taxa de acerto isolada não capturaria: uma criança pode estar acertando à custa de esforço excessivo, não de domínio confortável, e esse tipo de atrito é justamente o que o grafo precisa capturar para orientar a adaptação.

Fase 4 (continuação) — Construção do grafo com NetworkX

Com os pesos calculados, constrói-se o grafo propriamente dito: um nó para o aluno, um nó para cada conceito, e uma aresta entre eles com o peso de domínio calculado. Essa é a estrutura que, em produção, seria mantida para cada criança e atualizada continuamente por eventos de telemetria reais, em vez de dados de datasets públicos.

```
In [10]: # Eu defino a funcao que constroi o grafo de um aluno a partir da tabela
         # agregada de pesos
         def construir_grafo_aluno(id_aluno: str, tabela_conceitos: pd.DataFrame,
         coluna_conceito: str) -> nx.Graph:
             grafo = nx.Graph()

             # Eu adiciono o no do aluno
             grafo.add_node(id_aluno, tipo="aluno")

             # Eu adiciono um no por conceito e uma aresta ponderada entre o aluno e
             # cada conceito
             for _, linha in tabela_conceitos.iterrows():
                 conceito = str(linha[coluna_conceito])
                 grafo.add_node(conceito, tipo="conceito")
                 grafo.add_edge(
                     id_aluno,
                     conceito,
                     weight=linha["peso_dominio"],
                     n_interacoes=int(linha["n_interacoes"]),
                 )

             return grafo

         # Eu construo o grafo do aluno 14 usando os dados do ASSISTments
         grafo_aluno_14 = construir_grafo_aluno("aluno_14", aluno_14, "skill")

         print(f"Nos: {grafo_aluno_14.number_of_nodes()} | Arestas:
         {grafo_aluno_14.number_of_edges()}")
         print()
         for _, conceito, dados in grafo_aluno_14.edges(data=True):
             print(f"{conceito:15s} peso={dados['weight']:.3f} n_interacoes=
             {dados['n_interacoes']}")
```

Nos: 4 | Arestas: 3

Circle Graph	peso=0.411	n_interacoes=12
Median	peso=0.250	n_interacoes=4
Range	peso=0.750	n_interacoes=3

Interpretação

O grafo do aluno 14 tem 4 nós (o aluno e 3 conceitos) e 3 arestas, cada uma com o peso de domínio calculado na etapa anterior. Essa é a estrutura mínima que o motor de adaptação consultaria: dado um aluno, quais conceitos ele domina (peso alto) e quais precisam de adaptação mais cuidadosa (peso baixo). O mesmo código funciona para qualquer aluno da base, bastando trocar a fonte dos dados de entrada — dos datasets públicos utilizados aqui, para dados de telemetria real de uso, quando disponíveis.

Fase 4 (continuação) — Arestas entre conceitos

A estrutura construída até aqui é, tecnicamente, um grafo bipartido aluno-conceito: cada aluno se conecta aos conceitos com os quais interagiu, mas os conceitos não se

conectam entre si. Um Grafo de Conhecimento completo também precisa capturar a relação entre os próprios conceitos — por exemplo, se o domínio de um conceito tende a acompanhar o domínio de outro, isso é um indício de dependência ou proximidade cognitiva entre eles, informação que pode orientar por qual conceito começar uma intervenção.

A primeira tentativa foi construir essas arestas por co-ocorrência: dois conceitos seriam conectados se muitos alunos interagissem com ambos (índice de Jaccard sobre o conjunto de alunos de cada conceito). Essa abordagem se mostrou pouco discriminativa neste dataset — a mediana do índice de Jaccard entre todos os pares de conceitos já é 0,18, porque praticamente todos os alunos passam pelo mesmo currículo comum, então quase qualquer par de conceitos compartilha uma base grande de alunos. Um limiar baixo conectaria quase tudo; um limiar alto não distinguiria pares realmente relacionados dos que só compartilham currículo.

A alternativa adotada é a **correlação de desempenho entre conceitos**: para os alunos que interagiram com ambos os conceitos A e B, calcula-se a correlação entre suas taxas de acerto em A e em B. Se um aluno com baixo desempenho em A também tende a ter baixo desempenho em B, isso sugere uma demanda cognitiva compartilhada entre os dois conceitos — uma técnica estabelecida em mineração de dados educacionais para inferir estrutura de habilidades a partir de desempenho, sem depender de um currículo pré-anotado.

```
In [11]: # Eu construo a tabela aluno x conceito com a taxa de acerto de cada
         # combinacao,
         # usada para calcular a correlacao de desempenho entre pares de conceitos
         tabela_desempenho = interacoes.groupby(["user_id", "skill"])
         ["acerto"].mean().reset_index()
         pivot_desempenho = tabela_desempenho.pivot(index="user_id", columns="skill",
         values="acerto")

         # Eu mantenho so conceitos com pelo menos 20 alunos, para a correlacao nao ser
         # calculada
         # sobre uma base pequena demais para ser confiavel
         contagem_por_skill = pivot_desempenho.count()
         conceitos_validos = contagem_por_skill[contagem_por_skill >= 20].index
         pivot_filtrado = pivot_desempenho[conceitos_validos]

         print(f"Conceitos com pelo menos 20 alunos: {len(conceitos_validos)}")

         # Eu calculo a correlacao entre cada par de conceitos, exigindo pelo menos 15
         # alunos em comum
         # para o coeficiente ser considerado (min_periods evita correlacoes calculadas
         # sobre poucos pontos)
         correlacao_conceitos = pivot_filtrado.corr(min_periods=15)

         # Eu construo o grafo conceito-conceito, conectando pares com correlacao de
         # pelo menos 0.5
         LIMIAR_CORRELACAO = 0.5
         grafo_conceitos = nx.Graph()
         grafo_conceitos.add_nodes_from(conceitos_validos)

         # Eu pego so o triangulo superior da matriz de correlacao, para nao contar cada
         # par duas vezes
```

```

mascara_superior = np.triu(np.ones(correlacao_conceitos.shape),
k=1).astype(bool)
pares_superior = correlacao_conceitos.where(mascara_superior).stack()

for (conceito_a, conceito_b), valor in pares_superior.items():
    if valor >= LIMIAR_CORRELACAO:
        grafo_conceitos.add_edge(conceito_a, conceito_b, weight=round(valor,
3))

print(f"Nós: {grafo_conceitos.number_of_nodes()} | Arestas:
{grafo_conceitos.number_of_edges()}")
print(f"Componentes conexas:
{nx.number_connected_components(grafo_conceitos)}")
print(f"Nós isolados (sem par correlacionado acima do limiar): {sum(1 for n in
grafo_conceitos.nodes if grafo_conceitos.degree(n) == 0)}")

```

Conceitos com pelo menos 20 alunos: 87

Nós: 87 | Arestas: 75

Componentes conexas: 37

Nós isolados (sem par correlacionado acima do limiar): 36

Interpretação

Com o limiar de correlação de 0,5, o grafo resultante tem 87 nós e 75 arestas, organizados em 37 componentes conexas — inclusive 36 conceitos isolados, sem nenhum par correlacionado acima do limiar. Isso é, em si, um resultado interpretável: nem todo conceito tem uma relação forte de desempenho com outro conceito do currículo, e o grafo reflete essa realidade em vez de forçar conexões artificiais. A maior componente conexa reúne 51 conceitos, formando uma estrutura navegável o suficiente para os próximos passos (centralidade e caminho mínimo).

Centralidade: identificando conceitos-chave

Com arestas entre conceitos, é possível aplicar métricas clássicas de teoria de grafos. O **grau** (número de conexões) identifica conceitos com muitas relações; a **centralidade de intermediação** (betweenness) identifica conceitos que funcionam como ponte entre outras partes do grafo — removê-los desconectaria caminhos que passam por eles. Um conceito com alta centralidade de intermediação é candidato natural a ser abordado com prioridade, porque seu domínio facilita o acesso a vários outros conceitos conectados.

```

In [12]: # Eu calculo o grau de cada conceito (numero de conexoes diretas)
grau = dict(grafo_conceitos.degree())
top_grau = sorted(grau.items(), key=lambda item: -item[1][:8])

print("Top 8 conceitos por grau (mais conectados):")
for conceito, g in top_grau:
    print(f" {conceito:45s} grau={g}")

# Eu calculo a centralidade de intermediacao de cada conceito
centralidade_intermediacao = nx.betweenness_centrality(grafo_conceitos)
top_intermediacao = sorted(centralidade_intermediacao.items(), key=lambda item:
-item[1][:8])

print("\nTop 8 conceitos por centralidade de intermediação:")

```

```
for conceito, c in top_intermediacao:
    print(f" {conceito:45s} centralidade={c:.4f}")
```

Top 8 conceitos por grau (mais conectados):

Angles on Parallel Lines Cut by a Transversal	grau=11
Estimation	grau=8
Area Trapezoid	grau=7
Rounding	grau=7
Area Rectangle	grau=6
Circle Graph	grau=6
Complementary and Supplementary Angles	grau=6
Probability of a Single Event	grau=6

Top 8 conceitos por centralidade de intermediação:

Angles on Parallel Lines Cut by a Transversal	centralidade=0.1151
Conversion of Fraction Decimals Percents	centralidade=0.0791
Probability of a Single Event	centralidade=0.0763
Complementary and Supplementary Angles	centralidade=0.0734
Area Rectangle	centralidade=0.0689
Estimation	centralidade=0.0632
Area Triangle	centralidade=0.0579
Equation Solving Two or Fewer Steps	centralidade=0.0440

Interpretação

O conceito "Angles on Parallel Lines Cut by a Transversal" lidera tanto o grau (11 conexões) quanto a centralidade de intermediação (0,1151) — é o conceito mais estruturalmente central do grafo, funcionando como ponto de passagem entre diferentes grupos de conceitos. Isso não significa que ele seja o mais "importante" pedagogicamente em abstrato, mas sim que, dentro dos padrões de desempenho observados nesta base, ele está estatisticamente associado a um número maior de outros conceitos do que a média — um candidato natural para ser verificado com prioridade quando o objetivo é destravar o acesso a múltiplos conceitos relacionados.

Caminho mínimo entre conceitos

Duas noções de "caminho de menor atrito" combinam a estrutura do grafo conceito-conceito com o grafo aluno-conceito construído anteriormente: (1) o caminho mais curto entre dois conceitos, útil para saber por quais conceitos intermediários passar; e (2) cruzar essa estrutura com o perfil individual de um aluno, para verificar se os conceitos que ele domina menos são também os mais relacionados entre si — o que reforçaria a hipótese de uma dificuldade compartilhada, não coincidências isoladas.

```
In [13]: # Eu demonstro o caminho minimo entre dois conceitos que nao estao diretamente
         # usando a maior componente conexa do grafo
maior_componente = max(nx.connected_components(grafo_conceitos), key=len)
subgrafo_principal = grafo_conceitos.subgraph(maior_componente)

origem, destino = "Conversion of Fraction Decimals Percents", "Rounding"
distancia = nx.shortest_path_length(subgrafo_principal, origem, destino)
caminho = nx.shortest_path(subgrafo_principal, origem, destino)

print(f"Distancia entre '{origem}' e '{destino}': {distancia}")
```

```
print(f"Caminho: {' -> '.join(caminho)}")

# Eu cruzo a estrutura do grafo conceito-conceito com o perfil do aluno 14,
verificando
# se os dois conceitos onde ele tem pior desempenho (Median e Circle Graph) são
correlacionados
# entre si na base inteira de alunos
print()
print("Perfil do aluno 14 (da seção anterior):")
print(aluno_14[["skill", "peso_dominio"]].sort_values("peso_dominio"))

correlacao_median_circle = correlacao_conceitos.loc["Median", "Circle Graph"]
print(f"\nCorrelação de desempenho entre 'Median' e 'Circle Graph' na base
inteira: {correlacao_median_circle:.3f}")
```

Distância entre 'Conversion of Fraction Decimals Percents' e 'Rounding': 2
Caminho: Conversion of Fraction Decimals Percents -> Angles on Parallel Lines Cut by a Transversal -> Rounding

Perfil do aluno 14 (da seção anterior):

	skill	peso_dominio
1	Median	0.250
0	Circle Graph	0.411
2	Range	0.750

Correlação de desempenho entre 'Median' e 'Circle Graph' na base inteira: 0.456

Interpretação

Os dois conceitos "Conversion of Fraction Decimals Percents" e "Rounding" não são diretamente correlacionados acima do limiar, mas estão conectados por um caminho de distância 2, passando por "Angles on Parallel Lines Cut by a Transversal" — o mesmo conceito identificado como central na etapa anterior. Isso demonstra, de forma concreta, a utilidade de arestas conceito-conceito: uma consulta ao grafo pode revelar relações indiretas que não seriam visíveis olhando os conceitos isoladamente.

O cruzamento com o perfil do aluno 14 é o resultado mais relevante para o objetivo desta análise: os dois conceitos onde ele tem pior desempenho, **Median (peso 0,25)** e **Circle Graph (peso 0,41)**, apresentam correlação de desempenho de **0,456** na base inteira de alunos — o segundo maior valor entre todos os vizinhos de "Median". Isso sugere que a dificuldade desse aluno nos dois conceitos não é coincidência: eles compartilham uma demanda cognitiva comum na população em geral, o que reforça, com evidência estrutural do grafo, uma hipótese que a análise apenas do perfil individual não seria capaz de sustentar sozinha.

Fase 6 — Implantação: cold start e atualização contínua

O grafo construído acima utiliza dados históricos completos de um aluno — mas um aluno novo no sistema não tem histórico algum. A solução para esse problema é conhecida na literatura como **cold start**: o grafo nasce com um sinal grosseiro (por

exemplo, informações de perfil configuradas previamente) e é refinado progressivamente por eventos de uso real. A seguir, implementa-se a função de atualização incremental que resolve esse segundo estágio.

```
In [14]: # Eu defino a funcao que atualiza (ou cria) uma aresta do grafo a partir de um
         # unico evento novo,
         # em vez de recalcular tudo do zero – é assim que o grafo seria atualizado em
         # producao,
         # um evento de interacao de cada vez, nao em lote como feito acima so para
         # validar a tecnica
def atualizar_aresta_grafo(grafo: nx.Graph, id_aluno: str, conceito: str,
                          acertou: bool, latencia_ms: float) -> nx.Graph:
    # Eu calculo o peso do evento novo isoladamente
    velocidade_normalizada = 1 - min(latencia_ms, CAP_LATENCIA_MS) /
CAP_LATENCIA_MS
    peso_evento = (int(acertou) + velocidade_normalizada) / 2

    if grafo.has_edge(id_aluno, conceito):
        # Eu atualizo o peso existente com uma media movel simples, dando peso
        maior ao historico
        # (0.8) do que ao evento novo (0.2), para o grafo nao oscilar demais
        com um unico evento
        peso_atual = grafo[id_aluno][conceito]["weight"]
        novo_peso = round(0.8 * peso_atual + 0.2 * peso_evento, 3)
        grafo[id_aluno][conceito]["weight"] = novo_peso
        grafo[id_aluno][conceito]["n_interacoes"] += 1
    else:
        # Eu crio a aresta pela primeira vez (cold start para esse conceito
        especifico)
        grafo.add_node(conceito, tipo="conceito")
        grafo.add_edge(id_aluno, conceito, weight=round(peso_evento, 3),
n_interacoes=1)

    return grafo

# Eu simulo tres eventos novos chegando para o aluno 14 no conceito 'Median',
# onde ele tinha peso 0.25
print(f"Peso de 'Median' antes: {grafo_aluno_14['aluno_14']['Median']
['weight']}")

for _ in range(3):
    # Eu simulo o aluno acertando as proximas tentativas, com latencia razoavel
    grafo_aluno_14 = atualizar_aresta_grafo(grafo_aluno_14, "aluno_14",
"Median", acertou=True, latencia_ms=15000)

print(f"Peso de 'Median' depois de 3 acertos seguidos:
{grafo_aluno_14['aluno_14']['Median']['weight']}")
```

Peso de 'Median' antes: 0.25

Peso de 'Median' depois de 3 acertos seguidos: 0.555

Interpretação

O peso de 'Median' sobe de 0,25 para um valor mais alto depois de 3 eventos simulados de acerto, mas de forma gradual (média móvel com peso 0,8 para o histórico), não de uma vez só. Esse comportamento é intencional: um único acerto após uma sequência de

erros não deveria fazer o grafo concluir instantaneamente que o aluno passou a dominar o conceito — a atualização gradual evita instabilidade e reflete melhor a ideia de que domínio é construído ao longo de várias interações, não decidido em uma única tentativa.

Essa função fecha o ciclo de realimentação: cada evento de interação gerado pelo uso real chamaria `atualizar_aresta_grafo`, e a próxima adaptação para aquela criança já consultaria o grafo atualizado.

Síntese do ciclo CRISP-DM

Fase	Resultado
1. Negócio	Grafo de Conhecimento: nós = conceitos, arestas = domínio do aluno e relação entre conceitos, para orientar o motor de adaptação
2. Dados	ASSISTments (acerto e tentativas, 4.148 alunos) e EdNet (latência e tentativa de sinal de abandono, amostra)
3. Preparação	Explosão de listas em interações, agregação por aluno-conceito, extração de latência via timestamps, matriz aluno x conceito para correlação
4. Modelagem	Métrica de domínio por média de sinais normalizados; grafo bipartido aluno-conceito com NetworkX; grafo conceito-conceito por correlação de desempenho, com centralidade e caminho mínimo
5. Avaliação	Sinais isolados fracos (correlações entre -0,02 e -0,15); sinal de abandono não observável nesta amostra do EdNet; co-ocorrência de conceitos pouco discriminativa (mediana 0,18), correlação de desempenho mais informativa
6. Implantação	<code>atualizar_aresta_grafo()</code> define o contrato de atualização incremental do grafo aluno-conceito, com cold start e atualização gradual

Trabalhos futuros

- O sinal de abandono/persistência fina não está disponível nos datasets públicos testados — só existirá a partir de telemetria de uso real do sistema.
- O grafo conceito-conceito foi construído sobre correlação de desempenho, uma medida estatística que pode refletir tanto relação pedagógica genuína (como no par "Divisibility Rules" e "Greatest Common Factor", conceitos matematicamente relacionados) quanto proficiência geral do aluno em vez de uma dependência específica entre conceitos — essa distinção não foi isolada nesta análise e é uma limitação a considerar antes de usar o grafo para decisões automáticas de sequenciamento.
- O limiar de correlação (0,5) foi escolhido por inspeção da distribuição dos valores, não por validação externa; um estudo de sensibilidade a diferentes limiares fortaleceria a robustez da estrutura.
- A conexão dos conceitos deste grafo com dados de perfil configurados previamente (cold start) e com o domínio afetivo (medido por sinais fisiológicos) ainda não foi implementada.

- O campo 'part' foi usado como proxy de conceito no EdNet por falta de dicionário de tags nesta amostra pública; em um sistema em produção, os conceitos viriam de um vocabulário pedagógico definido explicitamente.